

```
In [1]: # E0: Native-language baseline
# XLM-RoBERTa fine-tuned directly on the low-resource dataset

import random
import numpy as np
import pandas as pd
import torch

from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    get_linear_schedule_with_warmup
)
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix
)
```

```
/home/jay/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
In [5]: # =====
# 1. Configuration
# =====

MODEL_NAME = "xlm-roberta-base"

DATA_PATH = "melayu_ternate.csv"

TEXT_COLUMN = "teks"
LABEL_COLUMN = "emosi"

MAX_LENGTH = 140
BATCH_SIZE = 16
EPOCHS = 5
LEARNING_RATE = 2e-5

RANDOM_SEED = 42

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", DEVICE)
```

Using device: cpu

```
In [6]: # =====
# 2. Reproducibility
# =====

def set_seed(seed):
    random.seed(seed)
```

```

np.random.seed(seed)
torch.manual_seed(seed)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)

set_seed(RANDOM_SEED)

```

```

In [7]: # =====
# 3. Load dataset
# =====

df = pd.read_csv(DATA_PATH)

# Remove rows with missing text/labels
df = df.dropna(subset=[TEXT_COLUMN, LABEL_COLUMN])

print("Number of examples:", len(df))
print("\nClass distribution:")
print(df[LABEL_COLUMN].value_counts())

```

Number of examples: 1399

Class distribution:

```

emosi
percaya      455
terkejut     286
marah        285
penyalahan   134
gembira      71
sedih        58
antisipasi   57
takut        40
jijik        13
Name: count, dtype: int64

```

```

In [8]: # =====
# 4. Encode labels
# =====

labels = sorted(df[LABEL_COLUMN].unique())

label2id = {
    label: i
    for i, label in enumerate(labels)
}

id2label = {
    i: label
    for label, i in label2id.items()
}

df["label_id"] = df[LABEL_COLUMN].map(label2id)

NUM_LABELS = len(labels)

print("\nLabels:")
print(label2id)

```

Labels:

```
{'antisipasi': 0, 'gembira': 1, 'jijik': 2, 'marah': 3, 'penyalahan': 4, 'percaya': 5, 'sedih': 6, 'takut': 7, 'terkejut': 8}
```

```
In [9]: # =====
# 5. Train / validation / test split
# =====

# First separate out the test set
train_val_df, test_df = train_test_split(
    df,
    test_size=0.20,
    random_state=RANDOM_SEED,
    stratify=df["label_id"]
)

# Then split the remaining 80% into
# 60% train and 20% validation
train_df, val_df = train_test_split(
    train_val_df,
    test_size=0.25,
    random_state=RANDOM_SEED,
    stratify=train_val_df["label_id"]
)

print("\nDataset sizes:")
print("Train:", len(train_df))
print("Validation:", len(val_df))
print("Test:", len(test_df))
```

Dataset sizes:

Train: 839

Validation: 280

Test: 280

```
In [10]: # =====
# 6. Tokenizer
# =====

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, timeout=120)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
In [11]: # =====
# 7. PyTorch Dataset
# =====

class EmotionDataset(Dataset):

    def __init__(self, dataframe, tokenizer, max_length):
        self.texts = dataframe[TEXT_COLUMN].tolist()
        self.labels = dataframe["label_id"].tolist()

        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, index):
```

```

        text = str(self.texts[index])
        label = self.labels[index]

        encoding = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_length,
            return_tensors="pt"
        )

        return {
            "input_ids": encoding["input_ids"].squeeze(0),
            "attention_mask": encoding["attention_mask"].squeeze(0),
            "labels": torch.tensor(label, dtype=torch.long)
        }

train_dataset = EmotionDataset(
    train_df,
    tokenizer,
    MAX_LENGTH
)

val_dataset = EmotionDataset(
    val_df,
    tokenizer,
    MAX_LENGTH
)

test_dataset = EmotionDataset(
    test_df,
    tokenizer,
    MAX_LENGTH
)

```

```

In [12]: # =====
# 8. DataLoaders
# =====

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)

```

```
In [13]: # =====
# 9. Model
# =====

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=NUM_LABELS,
    id2label=id2label,
    label2id=label2id
)

model.to(DEVICE)
```

```
Loading weights:  0%|          | 0/197 [00:00<?, ?it/s]
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 15692.89it/s]
[transformers] XLRobertaForSequenceClassification LOAD REPORT from: xlm-roberta-
base
```

Key	Status
lm_head.layer_norm.bias	UNEXPECTED
roberta.pooler.dense.bias	UNEXPECTED
lm_head.bias	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
roberta.pooler.dense.weight	UNEXPECTED
classifier.dense.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.bias	MISSING
classifier.out_proj.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

Out[13]: XLMRobertaForSequenceClassification(
  (classifier): XLMRobertaClassificationHead(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
    (out_proj): Linear(in_features=768, out_features=9, bias=True)
  )
  (roberta): XLMRobertaModel(
    (embeddings): XLMRobertaEmbeddings(
      (word_embeddings): Embedding(250002, 768, padding_idx=1)
      (token_type_embeddings): Embedding(1, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True, bias=
True)
      (dropout): Dropout(p=0.1, inplace=False)
      (position_embeddings): Embedding(514, 768, padding_idx=1)
    )
    (encoder): XLMRobertaEncoder(
      (layer): ModuleList(
        (0-11): 12 x XLMRobertaLayer(
          (attention): XLMRobertaAttention(
            (self): XLMRobertaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): XLMRobertaSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=Tru
e, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): XLMRobertaIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): XLMRobertaOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True,
bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
  )
)

```

```

In [14]: # =====
# 10. Optimizer and scheduler
# =====

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=LEARNING_RATE
)

total_training_steps = (
    len(train_loader) * EPOCHS
)

```

```

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=int(0.1 * total_training_steps),
    num_training_steps=total_training_steps
)

```

```

In [15]: # =====
# 11. Evaluation function
# =====

def evaluate(model, dataloader):

    model.eval()

    all_predictions = []
    all_labels = []

    total_loss = 0

    with torch.no_grad():

        for batch in dataloader:

            input_ids = batch["input_ids"].to(DEVICE)
            attention_mask = batch["attention_mask"].to(DEVICE)
            labels_batch = batch["labels"].to(DEVICE)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels_batch
            )

            total_loss += outputs.loss.item()

            predictions = torch.argmax(
                outputs.logits,
                dim=1
            )

            all_predictions.extend(
                predictions.cpu().numpy()
            )

            all_labels.extend(
                labels_batch.cpu().numpy()
            )

    average_loss = total_loss / len(dataloader)

    accuracy = accuracy_score(
        all_labels,
        all_predictions
    )

    macro_f1 = f1_score(
        all_labels,
        all_predictions,
        average="macro"
    )

```

```

    )

    weighted_f1 = f1_score(
        all_labels,
        all_predictions,
        average="weighted"
    )

    return {
        "loss": average_loss,
        "accuracy": accuracy,
        "macro_f1": macro_f1,
        "weighted_f1": weighted_f1,
        "labels": all_labels,
        "predictions": all_predictions
    }

```

```

In [16]: # =====
# 12. Training
# =====

best_val_f1 = -float("inf")
best_model_path = "best_e0_model.pt"

for epoch in range(EPOCHS):

    model.train()

    total_train_loss = 0

    for batch in train_loader:

        optimizer.zero_grad()

        input_ids = batch["input_ids"].to(DEVICE)
        attention_mask = batch["attention_mask"].to(DEVICE)
        labels_batch = batch["labels"].to(DEVICE)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels_batch
        )

        loss = outputs.loss

        loss.backward()

        # Prevent excessively large gradients
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            max_norm=1.0
        )

        optimizer.step()
        scheduler.step()

        total_train_loss += loss.item()

    average_train_loss = (

```

```

        total_train_loss / len(train_loader)
    )

    # Validation
    val_results = evaluate(
        model,
        val_loader
    )

    print(
        f"\nEpoch {epoch + 1}/{EPOCHS}"
    )

    print(
        f"Train loss: "
        f"{average_train_loss:.4f}"
    )

    print(
        f"Validation loss: "
        f"{val_results['loss']:.4f}"
    )

    print(
        f"Validation accuracy: "
        f"{val_results['accuracy']:.4f}"
    )

    print(
        f"Validation macro-F1: "
        f"{val_results['macro_f1']:.4f}"
    )

    print(
        f"Validation weighted-F1: "
        f"{val_results['weighted_f1']:.4f}"
    )

    # Save best model based on macro-F1
    if val_results["macro_f1"] > best_val_f1:

        best_val_f1 = val_results["macro_f1"]

        torch.save(
            model.state_dict(),
            best_model_path
        )

    print("Saved new best model.")

```

Epoch 1/5
Train loss: 2.0219
Validation loss: 1.7889
Validation accuracy: 0.3250
Validation macro-F1: 0.0545
Validation weighted-F1: 0.1594
Saved new best model.

Epoch 2/5
Train loss: 1.8034
Validation loss: 1.7674
Validation accuracy: 0.3321
Validation macro-F1: 0.0623
Validation weighted-F1: 0.1741
Saved new best model.

Epoch 3/5
Train loss: 1.7590
Validation loss: 1.7299
Validation accuracy: 0.3643
Validation macro-F1: 0.0992
Validation weighted-F1: 0.2464
Saved new best model.

Epoch 4/5
Train loss: 1.7295
Validation loss: 1.7264
Validation accuracy: 0.3786
Validation macro-F1: 0.1283
Validation weighted-F1: 0.2998
Saved new best model.

Epoch 5/5
Train loss: 1.6981
Validation loss: 1.7037
Validation accuracy: 0.3750
Validation macro-F1: 0.1335
Validation weighted-F1: 0.3076
Saved new best model.

```
In [17]: # =====  
# 13. Load best model  
# =====  
  
model.load_state_dict(  
    torch.load(  
        best_model_path,  
        map_location=DEVICE  
    )  
)
```

Out[17]: <All keys matched successfully>

```
In [18]: # =====  
# 14. Final test evaluation  
# =====  
  
test_results = evaluate(  
    model,  
    test_loader
```

```

)

print("\n=====")
print("FINAL E0 TEST RESULTS")
print("=====")

print(
    f"Accuracy:      "
    f"{test_results['accuracy']:.4f}"
)

print(
    f"Macro-F1:      "
    f"{test_results['macro_f1']:.4f}"
)

print(
    f"Weighted-F1:   "
    f"{test_results['weighted_f1']:.4f}"
)

```

```

=====
FINAL E0 TEST RESULTS
=====
Accuracy:      0.4214
Macro-F1:      0.1517
Weighted-F1:   0.3436

```

```

In [19]: # =====
# 15. Per-class results
# =====

print("\nClassification Report:")

print(
    classification_report(
        test_results["labels"],
        test_results["predictions"],
        target_names=labels,
        digits=4
    )
)

```

```

Classification Report:

```

	precision	recall	f1-score	support
antisipasi	0.0000	0.0000	0.0000	11
gembira	0.0000	0.0000	0.0000	14
jijik	0.0000	0.0000	0.0000	3
marah	0.4314	0.3860	0.4074	57
penyalahan	0.0000	0.0000	0.0000	27
percaya	0.4000	0.8352	0.5409	91
sedih	0.0000	0.0000	0.0000	12
takut	0.0000	0.0000	0.0000	8
terkejut	0.5128	0.3509	0.4167	57
accuracy			0.4214	280
macro avg	0.1494	0.1747	0.1517	280
weighted avg	0.3222	0.4214	0.3436	280

```

/home/jay/.local/lib/python3.10/site-packages/sklearn/metrics/_classification.py:
1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in la
bels with no predicted samples. Use `zero_division` parameter to control this beh
avior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/jay/.local/lib/python3.10/site-packages/sklearn/metrics/_classification.py:
1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in la
bels with no predicted samples. Use `zero_division` parameter to control this beh
avior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/jay/.local/lib/python3.10/site-packages/sklearn/metrics/_classification.py:
1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in la
bels with no predicted samples. Use `zero_division` parameter to control this beh
avior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

```

In [20]: # =====
# 16. Confusion matrix
# =====

cm = confusion_matrix(
    test_results["labels"],
    test_results["predictions"]
)

print("\nConfusion Matrix:")
print(cm)

```

```

Confusion Matrix:
[[ 0  0  0  0  0 11  0  0  0]
 [ 0  0  0  0  0 13  0  0  1]
 [ 0  0  0  0  0  2  0  0  1]
 [ 0  0  0 22  0 28  0  0  7]
 [ 0  0  0  4  0 22  0  0  1]
 [ 0  0  0  9  0 76  0  0  6]
 [ 0  0  0  1  0 10  0  0  1]
 [ 0  0  0  2  0  4  0  0  2]
 [ 0  0  0 13  0 24  0  0 20]]

```